

RLHF and PPO

pig7selene

September 5, 2026

Abstract

Reinforcement Learning from Human Feedback (RLHF) uses a learned preference signal to improve the behavior of an instruction-following language model beyond direct imitation. This chapter formulates an autoregressive language model as a token-level policy, derives the policy-gradient estimator used to optimize sequence-level rewards, and introduces the actor-critic and Generalized Advantage Estimation machinery that reduces its variance. It then develops Proximal Policy Optimization, distinguishes its clipped surrogate from the separate reference-model KL constraint used in language-model RLHF, and closes with the practical PPO loop, common failure modes, and implementation contracts.

1 Introduction

Chapter 13 established an SFT policy that imitates instruction-response demonstrations. Chapter 15 then showed how pairwise comparisons can train a Reward Model (RM) to rank complete responses. The next step is not another static supervised dataset update. It is an optimization problem in which a policy generates new responses, receives a score from the RM, and changes its next-token probabilities to increase expected reward while remaining close to useful SFT behavior.

The classical sequence is

$$\text{Pretraining} \rightarrow \text{SFT} \rightarrow \text{Reward Model} \rightarrow \text{RLHF} \rightarrow \text{PPO}. \quad (1)$$

In this chapter, **RLHF** denotes the policy-optimization stage that consumes the SFT initialization, a fixed or slowly refreshed RM, a prompt distribution, and a reference policy. **PPO** is the policy-gradient method used to make that optimization practical. The discussion is deliberately narrow: it supplies the reinforcement-learning notation needed for language-model post-training, but does not attempt a general reinforcement-learning textbook or introduce DPO and GRPO in depth.

2 Autoregressive Language Models as Policies

Let x be a prompt sampled from a declared prompt distribution $\mathcal{D}_{\text{prompt}}$. A generated response is $y = (a_0, \dots, a_{T-1})$, where each $a_t \in \mathcal{V}$ is a token. At generation step t , define the state as the prompt together with the generated prefix,

$$s_t = (x, a_0, \dots, a_{t-1}), \quad a_t \sim \pi_{\theta}(\cdot \mid s_t). \quad (2)$$

The policy π_{θ} is the language model's next-token distribution. The environment transition is almost deterministic: appending a_t produces s_{t+1} , except for stopping rules, truncation, and any external tool or simulator that a later application may introduce. A complete response is a trajectory, and its probability factorizes as

$$\pi_{\theta}(y \mid x) = \prod_{t=0}^{T-1} \pi_{\theta}(a_t \mid s_t). \quad (3)$$

In the common outcome-reward setting, the RM produces one scalar after a response is complete: $r_{\psi(x,y)}$. The model still takes one token action at a time, but the central quality signal is assigned at the end of the trajectory. This delayed, sampled reward is the source of both the flexibility and the difficulty of RLHF.

RL object	LLM realization	Role in the PPO stage
State s_t	Prompt plus all tokens generated before position t .	Conditions the next-token distribution and value estimate.
Action a_t	One sampled vocabulary token.	Receives a policy-gradient contribution through its log probability.
Trajectory y	A terminated or truncated assistant response.	Is scored by the RM and supplies token-level returns.
Policy π_θ	The trainable autoregressive language model.	Is improved from fresh rollouts using the clipped surrogate.
Reference π_{ref}	A fixed SFT checkpoint, commonly token-compatible with the policy.	Defines the KL anchor, not the rollout behavior policy.

Table 1: The RL abstraction preserves the autoregressive structure of a language model. The same completed response is a token trajectory for the policy and a sequence-level object for the Reward Model.

Table 1 exposes a crucial distinction. The RM is a function that scores a completed (x, y) pair. The policy is a conditional distribution that must choose every token along y . Backpropagating through the RM does not directly differentiate through discrete samples a_t or through the combinatorial set of possible continuations. Policy-gradient estimation supplies a way to improve the distribution from sampled trajectories instead.

3 Objective, Reward Shaping, and Policy Gradients

Let the unregularized objective be the expected RM score,

$$J_{\text{RM}}(\theta) = \mathbb{E}_{x \sim \mathcal{D}_{\text{prompt}}, y \sim \pi_{\theta}(\cdot | x)} [r_{\psi}(x, y)]. \quad (4)$$

Optimizing Equation 4 directly is difficult for three linked reasons. The response distribution changes when θ changes, a response contains many sampled discrete decisions, and RM scores are only approximations to the judgment one ultimately wants. A model that changes its distribution aggressively may visit responses on which the RM was never reliable. Classical language-model RLHF consequently regularizes the policy toward a fixed reference model, often the SFT checkpoint.

For each state, define the forward token-distribution divergence

$$\text{KL}_{t(\theta)} = \sum_{a \in \mathcal{V}} \pi_{\theta}(a | s_t) \log \frac{\pi_{\theta}(a | s_t)}{\pi_{\text{ref}}(a | s_t)}. \quad (5)$$

One conceptual regularized objective is

$$J(\theta) = \mathbb{E}_{x \sim \mathcal{D}_{\text{prompt}}, y \sim \pi_{\theta}(\cdot | x)} \left[r_{\psi}(x, y) - \beta \sum_{t=0}^{T-1} \text{KL}_{t(\theta)} \right], \quad \beta \geq 0. \quad (6)$$

The coefficient β controls the trade-off. A small value permits greater reward-driven deviation; a large value favors staying near the reference and can make learning nearly inert. The expected KL term is nonnegative, though a sampled per-token log-ratio estimate can be positive or negative. For a trajectory sampled by the behavior-policy snapshot π_{old} , many implementations use a sampled estimate to construct shaped rewards,

$$\tilde{r}_t^{\text{old}} = 1_{t=T-1} r_{\psi}(x, y) - \beta \log \frac{\pi_{\text{old}}(a_t | s_t)}{\pi_{\text{ref}}(a_t | s_t)}, \quad (7)$$

where the RM score is placed at the final response position and the KL cost is distributed across generated tokens. At collection time, π_{old} is the current policy snapshot, so Equation 7 estimates the objective in Equation 6. Once stored, these rewards, returns, and advantages are held fixed during PPO epochs; the changing π_θ then appears through the importance ratio. This changes credit assignment, not the intended trade-off: every token is charged for deviation from the reference, while the final response obtains the outcome score. Fine-Tuning Language Models from Human Preferences and InstructGPT are foundational examples of language-model reward optimization with a reference-model constraint [1], [2].

The log-derivative identity gives a policy-gradient estimator. Let $G_t^{\text{old}} = \sum_{k=t}^{T-1} \gamma^{k-t} \tilde{r}_k^{\text{old}}$ be the return from position t , with discount $\gamma \in [0, 1]$. At rollout collection, the estimator is

$$\nabla_{\theta} J(\theta) = \mathbb{E} \left[\sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) G_t^{\text{old}} \right]. \quad (8)$$

Equation 8 says that actions preceding a high return should become more likely, while actions preceding a low return should become less likely. It does not say that the last token alone caused the RM score. Every earlier token receives a shared, high-variance learning signal through its effect on the sampled continuation. The policy-gradient theorem formalizes this score-function route for parameterized policies [3].

3.1 Return, Value, and Advantage

The four quantities below serve different purposes. A **reward** $\tilde{r}_t^{\text{old}}$ is the immediate scalar assigned at one transition. A **return** G_t^{old} is the discounted sum of future rewards. The state value is the expected future return under a policy,

$$V^{\pi}(s_t) = \mathbb{E}[G_t^{\text{old}} | s_t]. \quad (9)$$

The action value $Q^{\pi}(s_t, a_t)$ is the expected return after taking a_t and following π thereafter. The **advantage** is their difference,

$$A^{\pi}(s_t, a_t) = Q^{\pi}(s_t, a_t) - V^{\pi}(s_t). \quad (10)$$

An advantage asks whether this action was better or worse than the policy’s expected continuation from the same state. Subtracting a state-only baseline such as $V^{\pi}(s_t)$ leaves the expected policy gradient unchanged while greatly reducing variance. Thus reward is the supplied signal, return is accumulated reward, value is a prediction of expected return, and advantage is a centered action-quality estimate. Treating them as interchangeable hides the function of the critic.

4 Actor-Critic Estimation and GAE

An actor-critic implementation learns both a policy and a value predictor. The **actor** is π_{θ} , which samples and assigns log probabilities to tokens. The **critic** is $V_{\varphi}(s_t)$, which predicts the shaped future return; it may use a separate model or a value head on a shared backbone. The critic is not the Reward Model. The RM maps a completed response to a preference-derived score and is normally frozen during a PPO update. The critic predicts the future reward that the current rollout process will produce and is trained from rollout returns.

For a sampled trajectory, define the temporal-difference residual

$$\delta_t = \tilde{r}_t^{\text{old}} + \gamma V_{\varphi}(s_{t+1}) - V_{\varphi}(s_t), \quad (11)$$

with $V_{\varphi}(s_T) = 0$ after termination. Generalized Advantage Estimation (GAE) forms

$$\hat{A}_t^{\text{GAE}} = \sum_{l=0}^{T-t-1} (\gamma\lambda)^l \delta_{t+l}, \quad \lambda \in [0, 1]. \quad (12)$$

When λ is near one, Equation 12 retains longer-horizon information and generally has less bootstrap bias but more variance. When it is near zero, it relies more heavily on the critic and has lower variance but more bias if the value estimate is inaccurate. Many language-model formulations use $\gamma = 1$ because a finite response is not naturally time-discounted; λ then still controls the bias-variance trade-off in the advantage estimator. GAE was introduced as a practical variance-reduction estimator for policy gradients [4].

The value function is commonly trained by regression to a declared target $\hat{G}_t$, for example a bootstrapped return compatible with the chosen GAE convention:

$$\mathcal{L}_{\text{value}(\varphi)} = \mathbb{E}_t[(V_{\varphi}(s_t) - \hat{G}_t)^2]. \quad (13)$$

Policy loss and value loss therefore have different targets. The policy loss changes action probabilities according to estimated advantage. The value loss teaches a predictor of shaped future return. A low

value loss does not prove that the policy is good; a high RM score does not prove that the critic is accurate.

5 From Importance Sampling to PPO

Rollouts are collected from a behavior policy π_{old} , a snapshot of the policy before the current optimization epoch. Once those samples have been stored, the probability ratio for their token actions is

$$r_{t(\theta)} = \frac{\pi_{\theta}(a_t | s_t)}{\pi_{\text{old}}(a_t | s_t)}. \quad (14)$$

Importance sampling uses Equation 14 to estimate how the new policy would weight actions sampled from the old policy. A simple surrogate is $\mathbb{E}[r_{t(\theta)} \hat{A}_t]$. It enables more than one minibatch pass over a rollout batch, but it becomes unreliable when the new policy moves far from the behavior policy: rare actions can receive large ratios, and a noisy advantage can drive a destructive update.

Proximal Policy Optimization replaces this surrogate with the clipped objective

$$\mathcal{L}^{\text{CLIP}(\theta)} = \mathbb{E}[\min(r_{t(\theta)} \hat{A}_t, \text{clip}(r_{t(\theta)}, 1-\varepsilon, 1+\varepsilon) \hat{A}_t)] . \quad (15)$$

For a positive advantage, increasing the probability ratio is useful only until it exceeds $1 + \varepsilon$; the clipped branch then prevents additional objective gain. For a negative advantage, decreasing the ratio is useful only down to $1 - \varepsilon$. The minimum in Equation 15 makes the surrogate pessimistic whenever the ratio leaves this interval in the direction that would otherwise improve the objective. PPO therefore constrains the incentive for one batch of noisy rollout data to induce a very large policy step, while still permitting several minibatch updates. PPO was introduced as a practical clipped surrogate for policy-gradient optimization [5].

5.1 PPO Clipping and Reference KL Are Not the Same

PPO clipping compares π_{θ} with π_{old} , the policy that generated the current on-policy rollouts. Its role is local: stabilize reuse of one rollout batch while the policy is updated. Reference KL compares π_{θ} with π_{ref} , the fixed SFT policy. Its role is global: preserve a behavioral anchor while the RLHF process moves across many rollout-and-update cycles.

Both mechanisms can limit drift, but neither replaces the other. A clipped update can still make steady, harmful movement away from SFT over many batches. A KL penalty alone can still permit an unstable optimization step relative to π_{old} . In language-model RLHF, the reference constraint also addresses a special problem: the RM is a learned proxy whose errors become easier to exploit as the policy searches beyond its training distribution. Chapter 15’s reward-overoptimization warning remains active throughout PPO.

6 The PPO Loop for Language Models

One RLHF iteration begins by drawing a batch of prompts, not by reusing a static preference pair as though it were an on-policy trajectory. The current policy generates Rollouts under fixed decoding and stopping rules. The RM scores each completed response, the reference model supplies token log probabilities, and the critic supplies values. The system constructs shaped rewards, returns, and advantages, then performs a limited number of PPO minibatch updates before discarding the rollout batch and sampling again.

Stage	Operation	State that must remain attached
1	Sample prompts	Prompt distribution revision, Chat Template, tokenizer, and prompt identifiers.
2	Generate Rollouts	Behavior-policy revision, token IDs, stop reason, sampled log probabilities, and decoding settings.
3	Score and shape reward	RM revision, reference log probabilities, KL coefficient, terminal-score convention, and reward normalization.
4	Estimate critic targets	Value predictions, masks, returns, advantages, and valid-token reductions.
5	Run PPO epochs	Old-policy log probabilities, clip range, policy and value losses, optimizer state, and minibatch schedule.
6	Evaluate and repeat	External evaluation, RM-score diagnostics, KL statistics, response length, and checkpoint identity.

Table 2: A PPO iteration is an on-policy dataflow, not supervised training on static preference pairs.

Each stored rollout needs the model revisions and token-level quantities that defined its objective. The policy, RM, value model, and reference model have different update schedules. The policy and critic usually change during PPO. The reference normally remains fixed for a run or a defined phase. The RM may remain frozen so that the objective is stable, or be refreshed only through a separately versioned preference-data procedure. Mixing these revisions without recording them makes a reward curve uninterpretable.

6.1 Stability, Sensitivity, and Reward Overoptimization

PPO reduces but does not eliminate sensitivity. A large policy learning rate, many epochs over one rollout batch, a broad clip range, or a poorly normalized advantage can cause unstable updates. An inaccurate critic makes $\hat{A}_t^{\text{GAE}}$ noisy or biased; a small rollout batch makes its variance high. Batch size changes both the statistical quality of the advantage estimate and the number of tokens through which the RM score is attributed. Chapters 7 and 8 remain relevant: optimizer settings, gradient clipping, precision, and finite-value checks still control the update path.

KL behavior supplies a useful but incomplete diagnostic. Persistently near-zero KL can indicate an overly strong β , a weak reward signal, or an update regime that is not moving the policy. Rapidly growing KL can indicate excessive drift, but a moderate aggregate KL does not prove that every behavior remains safe or useful. Adaptive KL control can target a declared range, yet it cannot compensate for an RM that ranks the wrong property.

Reward overoptimization is visible when RM reward rises while independent human or task-grounded evaluation stagnates or falls. Common mechanisms include response-length exploitation, polished but unresponsive verbosity, mode collapse toward a narrow high-scoring style, and exploitation of RM artifacts outside the preference-data distribution. The remedy is not simply more PPO clipping. It requires protected evaluation, diverse prompts and candidate distributions, RM audits, conservative policy updates, and explicit decisions about the desired trade-off. The language-model preference-learning literature documents that optimizing a learned reward can exploit labeler heuristics and proxy errors [1], [6].

7 Implementation Contracts

The rollout contract must version the prompt distribution, Chat Template, tokenizer, current policy checkpoint, sampling parameters, maximum response length, stop-token policy, and response parsing rule. It must retain token IDs, attention masks, action masks, sampled old-policy log probabilities, and the terminal condition. A prompt token is context, not an RL action; padding and post-termination positions must contribute to neither policy loss nor value loss.

The reward contract must name the RM revision, its sequence serialization and scalar-readout convention, the reference revision, the exact KL estimator, the coefficient β , reward normalization policy, and the terminal reward placement in Equation 7. Tests should confirm that reference log probabilities and RM scores are evaluated on exactly the response tokens generated by the behavior

policy. A changed Chat Template, stop convention, or response boundary changes the reward function operationally.

The optimization contract must distinguish π_{old} from π_{ref} , preserve the old log probabilities used in Equation 14, and record ε , γ , λ , PPO epoch count, minibatch construction, policy learning rate, value-loss coefficient, gradient controls, and valid-token denominators. It must declare whether actor and critic share parameters and how their losses are weighted. Diagnostic logs should include RM reward, shaped return, policy loss, value loss, advantage statistics, ratio and clip fractions, KL statistics, response length, entropy, finite-value flags, and independent evaluation results.

Finally, a resumable checkpoint requires more than policy weights. It must preserve the critic and any shared-head state, optimizer and scheduler states, RM and reference identities, prompt-data cursor, rollout-buffer semantics when resuming mid-iteration, random states, counters, and the full objective configuration. Chapter 11’s distinction between a model snapshot and a truly resumable training state applies unchanged to RLHF.

8 Summary

RLHF turns a preference-trained RM into an optimization signal for an autoregressive policy. A prompt plus generated prefix is the state, the next token is the action, and the completed response is a trajectory scored by the RM. Because the policy samples discrete tokens and reward is commonly delayed, policy gradients use sampled returns to change token probabilities. The critic supplies a value baseline, and GAE converts shaped rewards and value predictions into lower-variance advantage estimates.

PPO uses the old-policy probability ratio and a clipped surrogate to restrain each update on fresh Rollouts. The reference-model KL term serves a different purpose: it anchors the evolving policy to the SFT distribution and makes the reward-versus-drift trade-off explicit through β . Neither mechanism makes the RM correct. Policy loss, value loss, RM score, and external quality are distinct signals that must be monitored separately.

The practical loop is therefore a versioned on-policy system: sample prompts, generate responses, score them, estimate returns and advantages, update the policy and critic cautiously, evaluate externally, and repeat. Its central risk is reward overoptimization, in which a policy improves the learned proxy while degrading the intended behavior. The next post-training methods can be understood only after this distinction between preference-derived reward, policy optimization, and independent evaluation is clear.

References

- [1] D. M. Ziegler *et al.*, “Fine-Tuning Language Models from Human Preferences,” *arXiv preprint arXiv:1909.08593*, 2019, doi: 10.48550/arXiv.1909.08593.
- [2] L. Ouyang *et al.*, “Training Language Models to Follow Instructions with Human Feedback,” in *Advances in Neural Information Processing Systems 35*, 2022, pp. 27730–27744. doi: 10.52202/068431-2011.
- [3] R. S. Sutton, D. A. McAllester, S. P. Singh, and Y. Mansour, “Policy Gradient Methods for Reinforcement Learning with Function Approximation,” in *Advances in Neural Information Processing Systems 12*, MIT Press, 2000. [Online]. Available: <https://proceedings.neurips.cc/paper/1999/hash/464d828b85b0bed98e80ade0a5c43b0f-Abstract.html>
- [4] J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel, “High-Dimensional Continuous Control Using Generalized Advantage Estimation,” *arXiv preprint arXiv:1506.02438*, 2015, doi: 10.48550/arXiv.1506.02438.
- [5] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal Policy Optimization Algorithms,” *arXiv preprint arXiv:1707.06347*, 2017, doi: 10.48550/arXiv.1707.06347.
- [6] J. Eisenstein *et al.*, “Helping or Herding? Reward Model Ensembles Mitigate but Do Not Eliminate Reward Hacking,” *arXiv preprint arXiv:2312.09244*, 2023, doi: 10.48550/arXiv.2312.09244.